

CreatorOS

YouTube Data API v3 — Video Upload Integration

Technical Documentation for Google API Review

Prepared: Thu Jun 04 2026

1. Purpose of This Document

This document describes how CreatorOS uses the YouTube Data API v3 to upload video content on behalf of authenticated users. It is submitted in response to Google's request for a script or screencast demonstrating the upload flow, in support of OAuth 2.0 application verification.

CreatorOS is an autonomous media-business operating system for the ET Gaming 274 YouTube channel (@ETGAMING274). It automates: catalog import, AI-assisted clip selection, content scheduling, video upload, and performance learning — all operating within the caller's own YouTube account via OAuth 2.0.

2. OAuth 2.0 Authorization Flow

The application uses Google's OAuth 2.0 Authorization Code flow with offline access to obtain a refresh token, enabling background uploads without requiring the user to be present at the time of each upload.

2.1 Step-by-Step Flow

- User clicks "Connect YouTube Channel" in the CreatorOS dashboard.
- Server generates a cryptographically random state nonce (32 bytes, hex-encoded) and stores it in the session and database (oauth_nonces table, 10-min TTL).
- User is redirected to Google's authorization endpoint with the scopes below, accessType: "offline", and prompt: "consent".
 - After user grants permission, Google redirects to the registered callback URL (/api/auth/google/callback) with an authorization code and state parameter.
 - Server validates the state parameter against the stored nonce to prevent CSRF.
 - Server exchanges the authorization code for an access token and refresh token using the Google OAuth2 client library.
 - Tokens are stored encrypted in the PostgreSQL channels table, scoped to the authenticated user's account.
 - The access token is automatically refreshed every 12 hours via a background token guardian service, using the stored refresh token.

2.2 OAuth Scopes Requested

```
https://www.googleapis.com/auth/youtube
https://www.googleapis.com/auth/youtube.readonly
https://www.googleapis.com/auth/youtube.upload
https://www.googleapis.com/auth/youtube.force-ssl
https://www.googleapis.com/auth/youtube.channel-memberships.creator
https://www.googleapis.com/auth/youtubepartner
https://www.googleapis.com/auth/yt-analytics.readonly
https://www.googleapis.com/auth/yt-analytics-monetary.readonly
```

- youtube.upload — required

for `videos.insert` (video upload)

- `youtube.readonly` — read channel, playlist, and video metadata
- `youtube / youtube.force-ssl` — update video metadata, thumbnails, scheduling
- `yt-analytics.readonly` — read view counts, watch time, revenue estimates

2.3 OAuth Callback Endpoint

```
GET /api/auth/google/callback
! ' validates state nonce (CSRF guard)
! ' exchanges code for tokens via googleapis OAuth2Client
! ' persists access_token + refresh_token to channels table
! ' redirects user back to dashboard
```

3. Video Upload Flow (`videos.insert`)

All video uploads go through the `uploadVideoToYouTube()` function in `server/youtube.ts`. This function is the single upload entry point used by every upload path in the system.

3.1 Pre-Upload Checks

- Quota circuit breaker: if the YouTube API quota has been exhausted for the day (tracked in the `youtube_quota_usage` table), the upload is rejected with a clear error and deferred until midnight Pacific time.
- Daily upload cap: a per-user unit budget (1,600 units per `videos.insert` call) is checked before each upload to prevent exceeding the project quota.
- Token validation: the OAuth2 client is instantiated with the stored access token and refresh token; the `googleapis` library auto-refreshes if the access token is expired.

3.2 API Call — `videos.insert`

The upload uses the `googleapis` Node.js client library (v3). Key parameters:

```
const youtube = google.youtube({ version: "v3", auth: oauth2Client });

const response = await youtube.videos.insert({
  part: ["snippet", "status"],
  requestBody: {
    snippet: {
      title:          cleanTitle,          // "d 100 chars, banned phrases stripped
      description:    cleanDescription,    // "d 5,000 chars
      tags:           cleanTags,           // "d 500 chars total
      categoryId:     "20",                // Gaming
      defaultLanguage: "en",
      gameTitle:      "Battlefield 6",     // YouTube Studio "Game" field
    },
    status: {
      privacyStatus:  "private",           // "private" when scheduledStartTime set
      publishAt:      "<ISO-8601>",       // auto-switches to public at this time
      selfDeclaredMadeWithAI: false,       // YouTube 2025 AI disclosure (required)
```

3.3 Scheduling Logic

- If a future publish time is provided, `privacyStatus` is set to "private" and `publishAt` is set to the ISO-8601 timestamp.
- YouTube's own scheduler makes the video public at the specified time — no further API call is needed from the application.
- If the scheduled time is in the past, the video is published immediately at the requested privacy level.

3.4 Post-Upload Steps

- The returned YouTube video ID is stored in the `autopilot_queue` and `back_catalog_videos` tables for tracking.
- A pipeline-tracer background job verifies upload status via `videos.list` every 30 minutes to detect stuck or failed uploads.
- Quota units consumed are recorded in the `youtube_quota_usage` table.
- If the upload fails with a 401/invalid_grant error, the channel is flagged for reconnection and the user is notified via the dashboard.

4. What Is Uploaded and Why

CreatorOS exclusively operates on the authenticated user's own YouTube channel. No content is uploaded to third-party channels. All content originates from the user's own live-stream recordings.

Content types uploaded via `videos.insert`:

- YouTube Shorts ("d 60 seconds): clipped from the user's own live-stream VODs, using `yt-dlp` to extract segments from the user's own channel recordings.
- Long-form highlights (8–60 minutes): multi-segment extractions from streams longer than 60 minutes, split by natural content breaks.
- All video files are processed locally (`ffmpeg`) before upload. No third-party video sources are used.

5. Quota and Rate-Limit Management

- `videos.insert` costs 1,600 quota units per call.
- A circuit breaker (`youtube_quota_usage` table) trips at daily quota exhaustion and blocks all YouTube API calls until midnight Pacific time.
- Uploads are serialized and scheduled across the day: 3 Shorts/day + 1 long-form/day maximum, enforced by the output schedule service.
- Exponential back-off is applied on HTTP 429 and 500-series errors.
- Token refresh uses the OAuth2 refresh-token flow (no additional quota units).

6. Data Handling and Security

- OAuth tokens (`access_token`, `refresh_token`) are stored in a private PostgreSQL database scoped to the authenticated user's account.
- Tokens are never logged, never returned to the frontend, and never shared across users.
- All API calls are made server-side (Express backend). The browser never handles tokens.
- The state nonce used in the OAuth redirect is validated on callback and deleted from the database after a single use (CSRF protection).
- Users can revoke access at any time from Google Account settings or via the "Disconnect Channel" option in CreatorOS.

7. Step-by-Step Walkthrough (Screencast Script)

The following steps describe exactly what is shown in a live screencast demonstration of the upload

feature:

Step 1 — User Login

User logs in to CreatorOS via Replit OAuth. Session is established server-side.

Step 2 — Connect YouTube Channel

User navigates to Settings !' Channels !' "Connect YouTube". The app redirects to Google's consent screen requesting the scopes listed in Section 2.2.

Step 3 — Grant Permission

User selects their Google/YouTube account and clicks "Allow". Google redirects back to /api/auth/google/callback with the authorization code.

Step 4 — Token Exchange

The server exchanges the code for access + refresh tokens (server-side, not visible in browser). The channel appears as "Connected" in the dashboard.

Step 5 — Content Queue

User navigates to Content !' Autopilot. The system has automatically imported the back catalog and queued clips for upload. Scheduled items are visible with title, duration, and publish time.

Step 6 — Upload Trigger

At the scheduled time, the shorts-publisher or longform-publisher service calls `uploadVideoToYouTube()` (server/youtube.ts). The `videos.insert` API call is made with the parameters shown in Section 3.2.

Step 7 — Confirm in YouTube Studio

User opens YouTube Studio. The uploaded video appears with the correct title, description, tags, category (Gaming), and scheduled publish time. The "Game" field in Studio reflects the configured focus game.

Step 8 — Pipeline Verification

The Content !' Pipeline tab in CreatorOS shows the upload status verified via `videos.list`, confirming the video is live or scheduled on YouTube.

8. YouTube Data API v3 Endpoints Used

videos.insert: Upload video file with metadata and scheduling

videos.list: Verify upload status; read existing video metadata

videos.update: Update title, description, tags on existing videos

thumbnails.set: Upload custom thumbnail image

channels.list: Read channel info (name, subscriber count)

playlistItems.list: Import back-catalog video IDs from channel playlists

search.list: Discover existing channel content

liveBroadcasts.list: Detect active live streams (live copilot)

liveChatMessages.insert: Post copilot chat replies during live streams

youtubeAnalytics.query: Read view counts, watch-time, revenue estimates